Mastering how SPI flash memory actually works is a massive step up from basic Arduino programming. Flash memory (like your W25Q) does not behave like standard RAM or the built-in EEPROM you might be used to.
To use it efficiently, you have to understand the **"Erase Before Write"** rule.

1. The Rules of Flash Memory

Flash memory is divided into **Sectors** (usually 4,096 bytes) and **Pages** (usually 256 bytes).

- **The 0xFF Rule:** When memory is blank (erased), every bit is a 1 (which reads as 0xFF in hex). You can only write data by changing 1s to 0s.
- **You cannot change a 0 back to a 1 without erasing.**
- **Erasing is destructive and chunky:** You cannot erase a single byte. You must erase an entire 4KB Sector at once using flash.eraseSector(sectorNumber).

Efficient Workflow: Erase a sector *once*, then sequentially write small chunks of data to it (like Serial logs or arrays) until that 4KB sector is full.

2. Storing a Multidimensional Array

In C++, a multidimensional array (like int grid[3][4]) is actually just a single continuous block of bytes in memory. You don't need to save it number by number. You can take the entire array, cast it as a "byte pointer" (uint8_t*), and write the whole block to flash at once using sizeof(array).

3. Storing Runtime Serial Data

To store data on the fly without deleting your previous logs, we will:

1. Erase a designated sector (e.g., Sector 1, starting at address 4096) when the Arduino boots up.
2. Create a variable currentAddress that tracks exactly where the last write ended.
3. Every time a Serial message arrives, write it to currentAddress, then move currentAddress forward.

The Integrated Code

This code combines your RTC clock, writes a 2D array to Sector 0, and logs any text you type into the Serial Monitor sequentially into Sector 1, displaying the latest message on the screen.

```
#include <SPI.h>
#include <Arduino_GFX_Library.h>
#include <Adafruit_SPIFlash.h>
#include "RTC.h"

// --- DISPLAY SETUP ---
Arduino_DataBus *bus = new Arduino_UNOPAR8();
Arduino_GFX *tft = new Arduino_ILI9486(bus, A4, 0, false);

#define BLACK   0x0000
#define WHITE   0xFFFF
#define GREEN   0x07E0
```

```
#define CYAN    0x07FF
#define YELLOW  0xFFE0

// --- W25Q FLASH SETUP ---
#define FLASH_CS_PIN 10
Adafruit_FlashTransport_SPI flashTransport(FLASH_CS_PIN, &SPI);
Adafruit_SPIFlash flash(&flashTransport);

// Memory Addresses
const uint32_t ARRAY_ADDRESS = 0;       // Sector 0 (Address 0 to 4095)
uint32_t serialAddress = 4096;          // Sector 1 (Address 4096 to 8191)

// --- RTC SETUP ---
RTCTime currentTime;
int lastSecond = -1;

void setup() {
  Serial.begin(115200);

  // 1. Init Display
  tft->begin();
  tft->setRotation(1);
  tft->fillScreen(BLACK);

  // 2. Init RTC
  RTC.begin();
  RTCTime startTime(2, Month::SEPTEMBER, 2026, 14, 30, 00, DayOfWeek::WEDNESDAY,
SaveLight::SAVING_TIME_INACTIVE);
  RTC.setTime(startTime);

  // 3. Init Flash
  if (!flash.begin()) {
    Serial.println("Flash Init Failed!");
    return;
  }

  // ========================================
  // MULTIDIMENSIONAL ARRAY DEMO (SECTOR 0)
  // ========================================
  int sensorData[2][3] = {
    {12, 34, 56},
    {78, 90, 102}
  };

  flash.eraseSector(ARRAY_ADDRESS / 4096); // Erase Sector 0

  // Write the entire 2D array in one go
  flash.writeBuffer(ARRAY_ADDRESS, (uint8_t*)&sensorData, sizeof(sensorData));
```

```
  // Read it back into a new empty array to prove it worked
  int readData[2][3] = {0};
  flash.readBuffer(ARRAY_ADDRESS, (uint8_t*)&readData, sizeof(readData));

  // Display array on screen
  tft->setCursor(10, 10);
  tft->setTextColor(WHITE);
  tft->setTextSize(2);
  tft->print("Array[1][2]: ");
  tft->print(readData[1][2]); // Should print 102

  // ========================================
  // SERIAL LOGGING PREP (SECTOR 1)
  // ========================================
  // Erase Sector 1 once at startup so it's ready to accept sequential Serial writes
  flash.eraseSector(serialAddress / 4096);

  tft->setCursor(10, 60);
  tft->setTextColor(YELLOW);
  tft->print("Type in Serial Monitor...");
}

void loop() {
  // --- 1. RTC CLOCK UPDATE ---
  RTC.getTime(currentTime);
  if (currentTime.getSeconds() != lastSecond) {
    lastSecond = currentTime.getSeconds();

    char timeStr[10];
    sprintf(timeStr, "%02d:%02d:%02d", currentTime.getHour(), currentTime.getMinutes(),
currentTime.getSeconds());

    int textWidth = 8 * 2 * 6;
    tft->setCursor(tft->width() - textWidth - 10, 10);
    tft->setTextColor(CYAN, BLACK);
    tft->setTextSize(2);
    tft->print(timeStr);
  }

  // --- 2. RUNTIME SERIAL STORAGE ---
  if (Serial.available()) {
    String input = Serial.readStringUntil('\n');
    input.trim(); // Remove whitespace/newlines

    if (input.length() > 0) {
      // Write the string to the current free space in Sector 1
      // +1 ensures we save the null-terminator '\0' at the end of the string
```

```
      flash.writeBuffer(serialAddress, (uint8_t*)input.c_str(), input.length() + 1);

      // Read it back immediately to verify
      char readBack[64] = {0};
      flash.readBuffer(serialAddress, (uint8_t*)readBack, input.length() + 1);

      // Display what was just saved
      tft->fillRect(10, 90, 300, 30, BLACK); // Clear previous text area
      tft->setCursor(10, 90);
      tft->setTextColor(GREEN, BLACK);
      tft->print("Saved: ");
      tft->print(readBack);

      // Move the address pointer forward so the next write doesn't overwrite this one!
      serialAddress += (input.length() + 1);

      // Note: If serialAddress > 8191, you would need to erase Sector 2 and continue.
      // For this demo, we assume you won't type 4,000 bytes in one sitting.
    }
  }
}
```